

01 Project Overview & System Architecture

The Insurance Document Classifier is a browser-based ML application that classifies insurance documents (PDF, Word, Excel, scanned images) into a two-level hierarchy — Category → Subcategory — using a user-trained ML model. No external APIs, no cloud, no data leaves the machine.

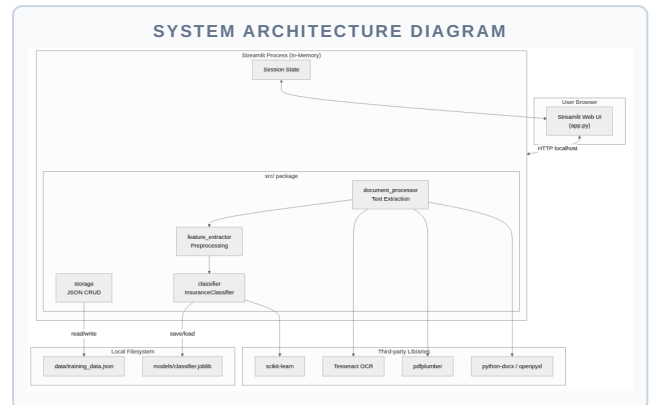
Capabilities

Capability	Detail
Ingest	PDF, DOCX, XLSX, PNG/JPG/TIFF/BMP/GIF
Extract	Native parsing for office formats; Tesseract OCR for images
Label	2-level: Category → Subcategory (e.g. Claims → Medical Records)
Train	4 scikit-learn algorithms, adaptive cross-validation
Predict	Top-5 ranked results with probability confidence scores
Persist	Training data → JSON · model → joblib (local disk only)

Technology Stack

Layer	Library
UI	Streamlit ≥ 1.35
ML	scikit-learn ≥ 1.4
PDF	pdfplumber ≥ 0.10
Office docs	python-docx · openpyxl · pandas
OCR	Pillow · pytesseract + Tesseract system binary
Visualization	matplotlib · seaborn

High-Level Architecture



Single-process Streamlit app — no separate backend, database server, or network calls. State lives in-process (session state) or on local disk (JSON + joblib).

src/ Package

Module	Responsibility
app.py	UI orchestration — Train, Predict, Model Info pages
document_processor	Format-agnostic text extraction (all file types)
feature_extractor	Text normalization + keyword boosting (×3 TF weight)
classifier	InsuranceClassifier — pipeline, train/predict/save/load
storage	CRUD over data/training_data.json
defaults	Built-in taxonomy: 6 categories, 6–8 subcategories each

Default Insurance Taxonomy

Category	Example Subcategories
Policy	Auto, Home/Property, Health, Life, Commercial
Claims	Claim Form, Medical Records, Adjuster Report
Underwriting	Application Form, Risk Assessment, Loss History
Financial	Premium Invoice, Payment Receipt, Audit Report
Legal	Endorsement, Exclusion Notice, Subrogation Letter
Correspondence	Cancellation Notice, Renewal Notice, Demand Letter

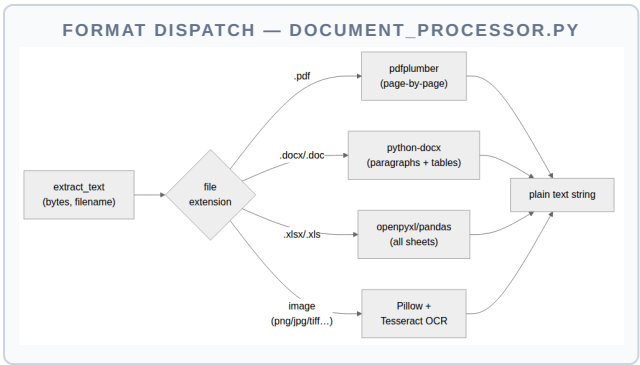
User-defined categories merge with defaults at runtime — no config needed.

Feature Engineering

Keyword Boosting: Keywords appended 3× to inflate TF-IDF weight — lightweight human-in-the-loop signal with no custom vectorizer needed.

```
processed = lowercase + strip_non_alphanum + collapse_ws
feature    = processed + " " + " ".join(keywords) * 3
```

Document Processing Pipeline

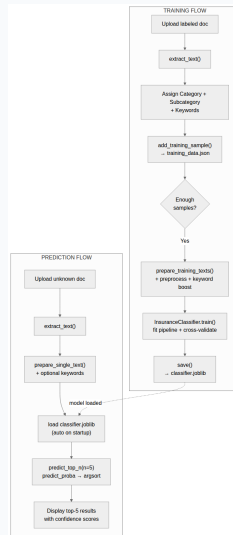


PDF: pdfplumber page-by-page. **DOCX:** paragraphs + table cells (" | "). **XLSX:** all sheets linearized to rows. **Images:** Pillow → Tesseract OCR.

Directory Structure

```
insurance-doc-classifier/
├─ app.py                # Entry point & UI
├─ requirements.txt
├─ src/
│  ├─ classifier.py      # ML engine
│  ├─ defaults.py        # Taxonomy
│  ├─ document_processor.py
│  ├─ feature_extractor.py
│  └─ storage.py
├─ data/training_data.json # Auto-created
└─ models/classifier.joblib # Auto-created
```

END-TO-END DATA FLOW — TRAINING (TOP) AND PREDICTION (BOTTOM)



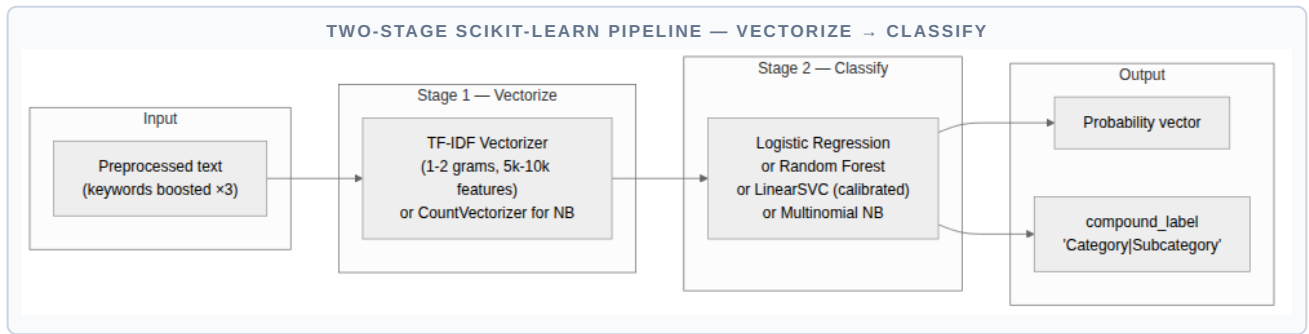
Training Flow

#	Step	Function
1	Upload + extract text	<code>extract_text(bytes, filename)</code>
2	Assign label + keywords	UI selection
3	Persist sample	<code>add_training_sample()</code> → JSON
4	Batch-prepare features	<code>prepare_training_texts()</code>
5	Fit pipeline	<code>InsuranceClassifier.train()</code>
6	Cross-validate	<code>cross_val_score()</code> adaptive folds
7	Serialize model	<code>save()</code> → <code>classifier.joblib</code>

Prediction Flow

#	Step	Function
1	Auto-load model on startup	<code>InsuranceClassifier.load()</code>
2	Extract text from doc	<code>extract_text(bytes, filename)</code>
3	Preprocess + boost keywords	<code>prepare_single_text()</code>
4	Rank top-5 predictions	<code>predict_top_n(n=5)</code>
5	Parse compound label	<code>_parse_result()</code> splits on <code> </code>
6	Display confidence	Green ≥70% · Orange ≥40% · Red <40%

Compound labels: "Claims|Medical Records" is the single label the model learns. `_parse_result()` splits on `"|"` post-prediction to recover both hierarchy levels.



Algorithm Comparison

Property	Logistic Regression	Random Forest	SVM Linear	Naive Bayes
Vectorizer	TF-IDF	TF-IDF	TF-IDF	CountVectorizer
Max features	10,000	5,000	10,000	10,000
N-grams	(1,2)	(1,2)	(1,2)	(1,2)
predict_proba	Native	Native	Platt scaling (CalibratedClassifierCV)	Native
Parallelism	—	n_jobs=-1	—	—
Best for	General baseline	Larger datasets	High-dimensional text	Very small datasets

Cross-Validation

```

cv_folds = min(5, min_class_count)
cross_val_score(pipeline, texts, labels, cv=cv_folds)
training_accuracy = cv_scores.mean()
  
```

Folds capped at min class count — no empty test splits. Falls back to training accuracy when too few samples.

SVM calibration: LinearSVC wrapped in CalibratedClassifierCV(cv=3) (Platt scaling) to expose predict_proba.

Model Serialization & Deployment

```

payload = {
    "pipeline": Pipeline, # fitted sklearn Pipeline
    "classifier_name": str,
    "training_accuracy": float,
    "cv_scores": ndarray | None,
    "classes_": list[str],
    "n_samples_trained": int
}
  
```

```

pip install -r requirements.txt
# OCR (optional): brew install tesseract
# sudo apt-get install tesseract-ocr
streamlit run app.py # → http://localhost:8501
  
```

Training Sample (JSON Schema)

```
{
  "id": "uuid-v4",
  "filename": "claim_form.pdf",
  "text": "extracted full text...",
  "category": "Claims",
  "subcategory": "Medical Records",
  "keywords": ["medical", "claim number"],
  "compound_label": "Claims|Medical Records",
  "timestamp": "2026-06-22T14:23:01"
}
```

Flat JSON array in `data/training_data.json`. Only `storage.py` reads or writes this file.

Three-Page UI

Page	Key Features
Train	Upload → assign category/subcategory + keywords → add to dataset → select algorithm → train → accuracy & CV scores
Test/ Predict	Upload → optional keywords → classify → top-5 results → confidence bar (green/orange/red)
Model Info	Metrics → class distribution chart → confusion matrix → CSV export

Error Handling

Scenario	Handling
Unsupported file type	<code>ValueError</code> shown as <code>st.error</code>
Tesseract not installed	<code>RuntimeError</code> with install instructions
Fewer than 2 samples	Train button disabled with UI warning
Corrupt model file	<code>try/except</code> → <code>None</code> , sidebar shows warning

Session State

Key	Purpose
<code>classifier</code>	Active trained model (auto-loaded on startup)
<code>train_text</code>	Extracted text from training upload
<code>test_text</code>	Extracted text for prediction

Extension Points

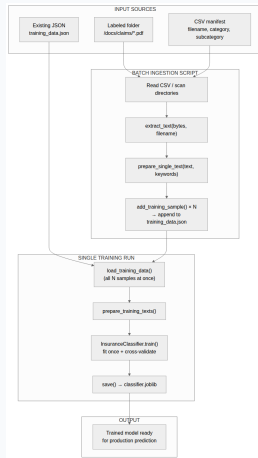
Extension	File to change
New ML algorithm	<code>classifier.py</code> — <code>_build_pipeline()</code> + <code>ALGORITHM_NAMES</code>
New file format	<code>document_processor.py</code> — <code>SUPPORTED_EXTENSIONS</code>
Database backend	<code>storage.py</code> only — public API unchanged
REST API	Wrap <code>classifier.predict</code> + <code>extract_text</code> in FastAPI
LLM classifier	Implement same <code>train/predict/save/load</code> interface

Project Structure

```
insurance-doc-classifier/
├─ app.py · requirements.txt
├─ src/ (classifier · defaults
│   │   document_processor
│   │   feature_extractor · storage)
├─ data/training_data.json
├─ models/classifier.joblib
```

The UI processes one document at a time — fine for small datasets, but slow for hundreds of pre-labeled files. Because `src/` modules are fully importable as a Python library, a batch script can ingest an entire labeled archive and trigger **one single training run**, eliminating all redundant intermediate pipeline fits.

BULK TRAINING FLOW — CSV / FOLDER → SINGLE TRAINING RUN



Why Batching Saves Time & Cost

Approach	Training runs	100-doc effort
UI — one by one	Up to 100	~30–60 min manual
Batch script	1	~10–30 sec automated

Every `InsuranceClassifier.train()` re-fits the full pipeline from scratch. Batching all samples first then training once avoids N redundant fits and cross-validation runs.

Input Options

Input	When to use
Labeled folder tree	Files organized by category in directories
CSV manifest	Labels in a spreadsheet, files in flat folder
Existing JSON	Migrating or merging from another instance

Best practice: aim for ≥ 10 samples per class. Check mean CV accuracy after training — below 70% signals more or better-labeled data is needed.

Batch Ingestion Script (CSV)

```
import csv, pathlib, sys
sys.path.insert(0, ".")
from src.document_processor import extract_text
from src.storage import (add_training_sample,
                        clear_all_training_data, load_training_data)
from src.classifier import InsuranceClassifier
from src.feature_extractor import
prepare_training_texts
```

```
DOCS_DIR = pathlib.Path("docs/")
MANIFEST = "labels.csv"
# CSV: filename, category, subcategory, keywords
```

```
clear_all_training_data() # optional: start fresh
```

```
with open(MANIFEST, newline="") as f:
    for row in csv.DictReader(f):
        data = (DOCS_DIR /
                row["filename"]).read_bytes()
        text = extract_text(data, row["filename"])
        kws = [k.strip() for k in
                row.get("keywords", "").split("|") if k]
        add_training_sample(
            text=text,
            filename=row["filename"],
            category=row["category"],
            subcategory=row["subcategory"],
            keywords=kws,
        )
```

```
# One single training run across all N samples
samples = load_training_data()
texts = prepare_training_texts(samples)
labels = [s["compound_label"] for s in samples]
clf = InsuranceClassifier("Logistic Regression")
clf.train(texts, labels)
clf.save("models/classifier.joblib")
print(f"Accuracy: {clf.training_accuracy:.1%}"
      f" | Samples: {clf.n_samples_trained}")
```

CSV Manifest Format

```
filename,category,subcategory,keywords
claim_q1.pdf,Claims,Medical Records,medical|diagnosis
policy_auto.docx,Policy,Auto Policy,premium|vehicle
invoice_mar.xlsx,Financial,Premium Invoice,invoice|due
```